Using Curvature

Newton, L-BFGS, K-FAC, Shampoo & Sophia

Lesson 7 of the Optimizer Course

The gold-standard methods that use the *real* curvature — and why we usually can't afford them

Every optimizer so far used only the slope, and merely guessed *at curvature. This lesson uses the real thing — the Hessian from Lesson 1. The reward is spectacular: Newton's method reaches the exact bottom of our bowl in a single step. The price is brutal: the Hessian is far too big to compute. So this lesson is really about the clever approximations — L-BFGS, K-FAC, Shampoo, Sophia — that try to buy most of the reward for a fraction of the price.*

Marevlo Research

1 The dream: Newton's method

Back in Lesson 1 we learned that near any point the loss looks like a bowl, described by the second-order Taylor expansion — and the Hessian H *is* that bowl's exact shape. So why inch downhill at all? Why not jump straight to the bottom of the local bowl? That jump is **Newton's method**:

$$w \leftarrow w - H^{-1}\nabla f$$

Say it in English: instead of stepping along the raw slope, first "undo" the curvature by multiplying by H^{-1}. That bends the step so it points at the true bottom, not just downhill.

The difference from gradient descent is the H^{-1} in front. Gradient descent uses a fixed scalar step η for every direction; Newton uses the inverse curvature, which is a *custom* step for every direction at once.

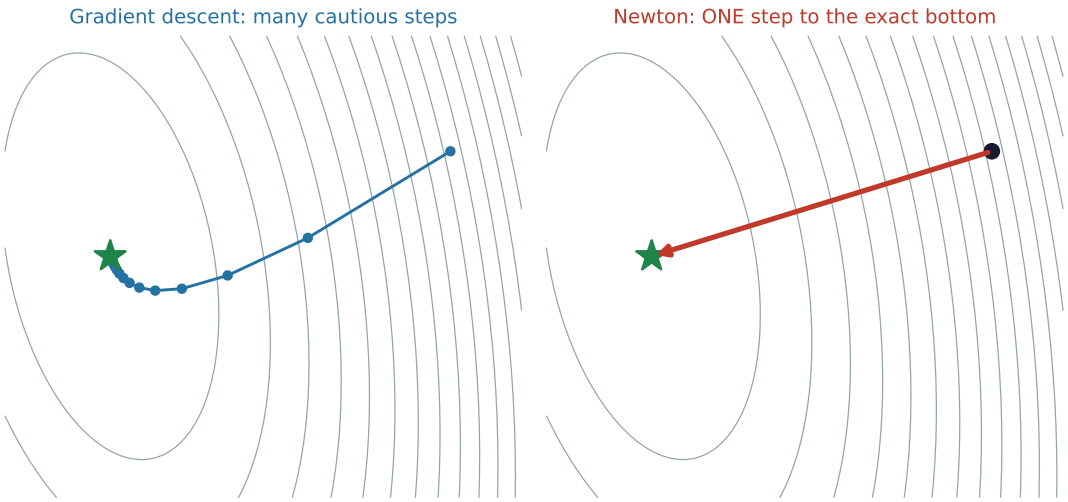
Let's actually do the numbers

Take the friendly bowl from Lesson 1: $H = A = \left[\begin{smallmatrix} 4 & 1 \\ 1 & 3 \end{smallmatrix}\right]$, $\nabla f = Aw - b$, true bottom $w^* = (0.091,\ 0.636)$. Start anywhere — say $w_0 = (2, 1)$, where $\nabla f = (8, 3)$. The inverse Hessian is $A^{-1} = \frac{1}{11}\left[\begin{smallmatrix} 3 & -1 \\ -1 & 4 \end{smallmatrix}\right]$. One Newton step:

$$H^{-1}\nabla f = \frac{1}{11}\begin{bmatrix} 3 & -1 \\ -1 & 4 \end{bmatrix}\begin{bmatrix} 8 \\ 3 \end{bmatrix} = \frac{1}{11}\begin{bmatrix} 21 \\ 4 \end{bmatrix} = (1.909,\ 0.364).$$

$$w_0 - H^{-1}\nabla f = (2, 1) - (1.909,\ 0.364) = (0.091,\ 0.636) = w^*.$$

The exact bottom, in one step. Gradient descent needed dozens of steps to merely approach it. (Why exact? Because our bowl *is* a perfect quadratic, and Newton solves a quadratic in one move. On a real curvy loss it'd take a few steps, but still far fewer than first-order methods.)



There's a deeper point here. Multiplying by H^{-1} rescales each natural direction by $1/\lambda$ — so the steep directions (big λ) get shrunk and the shallow ones (small λ) get stretched, until *every direction is balanced.* Newton effectively turns the condition number κ into 1. **Every adaptive optimizer we've studied — Adagrad, RMSProp, Adam — was a cheap attempt to**

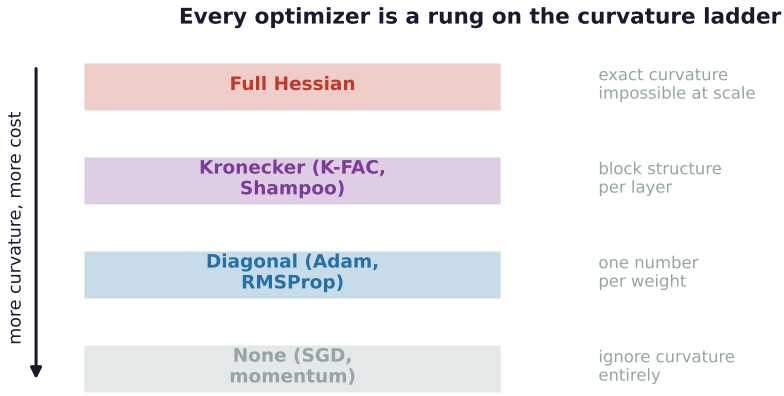
approximate exactly this. Newton is the real thing they were all imitating.

2 Why we can't just use Newton

If Newton is so good, why isn't it the default? Because H is an $n \times n$ matrix.

- For a model with $n = 1$ billion weights, H has 10^{18} entries — you cannot store it, let alone compute it.
- *Inverting* an $n \times n$ matrix costs about n^3 operations. For a billion parameters that's 10^{27} operations per step. Hopeless.

So the entire field of second-order optimization is one question: **how do we get the benefit of H^{-1} without ever building H?** The next methods are four different answers.



3 L-BFGS: infer curvature from history

L-BFGS never forms H. Instead it watches how the gradient *changes* as you move and infers the curvature from that. The intuition is a single fact: if the gradient changed by Δg when you moved by Δw, then the curvature in that direction is roughly $\Delta g / \Delta w$ (that's literally what a second derivative is — a change in slope per change in position).

By remembering just the last ∼10–20 (move, gradient-change) pairs, L-BFGS reconstructs the effect of H^{-1} on the gradient without ever storing a matrix. The "L" is for "limited-memory." It's excellent for **smooth, full-batch problems**: classical machine learning, logistic regression, physics-informed networks. Its weakness is minibatch noise — the curvature inference needs consistent gradients, and SGD's randomness scrambles it, which is why you rarely see L-BFGS on big stochastic deep-learning runs.

4 K-FAC and Shampoo: factor the curvature

These two bring second-order methods to actual deep networks by exploiting *structure*. The key trick is the **Kronecker product**: a big matrix that happens to be built from two small ones can be stored and inverted via just the small ones.

Let's actually do the numbers

A 4×4 curvature block that factors as $A \otimes B$ (two 2×2 matrices):

$$A \otimes B = \begin{bmatrix} 2 & 2 & 0 & 0 \\ 2 & 4 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 2 \end{bmatrix} \quad \text{is fully described by} \quad A = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}, \; B = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}.$$

Store A and B: $4 + 4 = 8$ numbers instead of 16. The win explodes with size: for a layer with 1000 inputs and 1000 outputs, the factors hold 2,000,000 numbers versus the full block's 10^{12} — about **500,000× smaller**, and you invert two 1000×1000 matrices instead of one $10^6 \times 10^6$ monster.

K-FAC uses exactly this: it approximates each layer's curvature as a Kronecker product of two small matrices — one built from the layer's inputs, one from its output gradients — and inverts those. It's a practical approximation to the "natural gradient," which steps in the geometry of the function the network computes rather than raw weight space.

Shampoo takes the same Kronecker idea further, maintaining a preconditioner for each axis of each weight tensor — think of it as "Adagrad, but with small matrices instead of single numbers per weight," so it can capture *correlations* between parameters that diagonal Adam is blind to. With careful engineering (the matrix roots are the costly part), Shampoo has become genuinely competitive at **LLM scale**, used in some large Google training runs.

5 Sophia: second-order, made cheap enough

Sophia (2023) is a recent attempt to make second-order practical for LLM pretraining specifically. It uses a *cheap, diagonal* estimate of the Hessian as a preconditioner — and crucially only refreshes that estimate every ∼10 steps, so the second-order overhead is amortised to almost nothing. It then *clips* the per-coordinate update so a bad curvature estimate can't cause a wild step. The reported payoff is roughly a **2× speedup over AdamW** on language-model pretraining at similar cost per step. It's promising and actively used in research, though not yet the universal default.

When you're actually training

Reality check: for everyday training, first-order **AdamW still wins** on simplicity and robustness, and that's what you should use. Second-order methods earn their keep in specific places: **L-BFGS** for small/full-batch and scientific computing; **K-FAC / Shampoo** when someone has invested the engineering for a big run and wants faster convergence per step; **Sophia** for LLM pretraining if you're chasing wall-clock speedups. The honest summary: they offer fewer, better-aimed steps, in exchange for much more cost and complexity per step.

6 What you can do now, and what's next

You can now write Newton's update and explain why H^{-1} gives the perfect step, compute a Newton step by hand (one step to the bottom), explain precisely why the full Hessian is unaffordable, and describe how L-BFGS, K-FAC, Shampoo, and Sophia each approximate

curvature cheaply.

Next — Lesson 8: Generalizing Well. Every method so far chased the *lowest* point on the training hill. But here's the twist that decides whether your model actually works on new data: two bottoms at the same height aren't equally good. A *wide, flat* valley generalises far better than a *narrow, sharp* one. Lesson 8 is about optimizers — **SAM** and **Lookahead** — that deliberately seek flat bottoms, not just low ones.